

Lesson Activities — 1.3 Exchanging Data

A bank of low-prep, in-lesson classroom activities for OCR A Level Computer Science (H446) Section 1.3 — designed for active teaching, not assessment; most need only mini-whiteboards, envelopes or scrap card.

1.3.1 Compression, Encryption & Hashing

Human Run-Length Encoding (unplugged human demonstration · 10 min · whole class)

Resources: Zero-prep (optional: two colours of paper for "pixels") **How it runs:** 1. Line up 17 students at the front as a "row of pixels": 6 holding/wearing A, then 3 B, then 8 C (or use two colours: 6 white, 3 black, 8 white). 2. Ask the class: "How could we describe this row to someone next door using the fewest characters?" Take suggestions. 3. Steer to RLE: one student walks the line and calls out the encoding; a scribe writes it on the board. 4. Now rearrange the same students into the pattern A B C A B C A B C... and repeat. Ask: is RLE still a good idea? 5. Debrief: when does RLE shrink a file, and when does it inflate one? **Answers / expected outcomes:** Row 1: A6 B3 C8 — 17 characters compressed to 6 (assuming one char per value + one per count). Row 2: A1 B1 C1 A1 B1 C1 ... — every alternating pattern doubles in size (e.g. ABCABC, 6 chars, becomes A1B1C1A1B1C1, 12 chars). Key point: RLE is lossless but only reduces size when there are long runs of repeated values; with few repeats it can *enlarge* the file.

Stretch: Ask students to encode the same rows with dictionary compression (pattern ABC stored once, referenced 3 times) and compare which wins on each row.

Padlocks & Caesar Wheels: Symmetric vs Asymmetric (unplugged encryption demo · 15 min · whole class)

Resources: Two envelopes/boxes; optional real padlock; board **How it runs:** 1. **Symmetric round:** Two volunteers (Alia and Ben) sit at opposite ends of the room. Alia must send Ben the secret word REVISE. Agree publicly that the algorithm is a Caesar cipher; Alia and Ben share the key "shift +3" (whisper it to both). Alia encrypts on a mini-whiteboard, passes the ciphertext through 3–4 "interceptor" students, Ben decrypts by shifting back 3. 2. Ask the interceptors what they saw. Then ask the killer question: "How did Alia and Ben agree on the key +3 in the first place, without the interceptors hearing?" Let the class hit the key distribution problem. 3. **Asymmetric round:** Ben now hands out *open padlocked boxes* (or envelopes labelled "Ben's public key — anyone may lock") to the whole class, but keeps the only key (private key) in his pocket. Alia puts her message in a box, snaps it shut, and sends it through the same interceptors. Only Ben can open it. 4. Debrief on the board: same key both ways = symmetric (fast, key distribution problem); public key locks / private key unlocks = asymmetric (slower, solves distribution). 5. Finish by explaining that real systems (e.g. HTTPS) use asymmetric encryption just to share a symmetric session key, then switch to symmetric for speed. **Answers / expected outcomes:** Caesar +3 of REVISE = UHYLVH (R → U, E → H, V → Y, I → L, S → V, E → H). Interceptors see only ciphertext but *could* decrypt if they overheard the key — the weakness of symmetric encryption. In the padlock round interceptors can lock boxes but never open them: encrypting with the public key means only the private key holder can decrypt. Expected vocabulary in debrief: plaintext, ciphertext, key, key distribution problem, public/private key pair. **Stretch:** Ask how Ben could prove a message really came from him (sign with private key, anyone verifies with public key — digital signature).

One-Way Street: Human Hash Function (unplugged demonstration · 10 min · whole class)

Resources: Zero-prep; board **How it runs:** 1. Define a toy hash function on the board: $\text{hash}(\text{word}) = (\text{number of letters}) \bmod 5$. 2. Students hash their own first name on mini-whiteboards and hold up the digit. 3. Ask: "I'm thinking of a word whose hash is 1 — what was my word?" (Impossible — one-way.) 4. Find two students with the same hash value but different names — name this a **collision**. 5. Hash the networking words **CACHE**, **ROUTER**, **SWITCH** together, then debrief: which properties of a good hash does our toy function have (deterministic, fixed-length output, fast) and which does it lack (far too many collisions)? 6. Link to uses: password storage (store the hash, compare hashes at login) and hash tables. **Answers / expected outcomes:** **CACHE** (5 letters) → 0; **ROUTER** (6) → 1; **SWITCH** (6) → 1 — **ROUTER** and **SWITCH** collide. Students should articulate: hashing is one-way (cannot recover the input), deterministic (same input → same output), fixed-length output, and collisions should be rare in a good function. Crucial distinction: hashing ≠ encryption because encryption is reversible with a key. **Stretch:** Ask why a system should hash passwords rather than encrypt them (even the administrator/attacker with database access cannot reverse a hash; no decryption key exists to steal).

Odd One Out: Compression & Crypto (odd one out + think-pair-share · 10 min · pairs)

Resources: Board or slide with the four sets; mini-whiteboards **How it runs:** 1. Display four sets. Pairs get 60 seconds per set: pick an odd one out and write *why* on a whiteboard. 2. Sets: (a) **JPEG** · **MP3** · **PNG** · video streaming; (b) **RLE** · dictionary coding · **JPEG** · **ZIP**; (c) public key · private key · session key · hash digest; (d) Caesar cipher · one shared secret key · fast bulk encryption · key pair. 3. Take answers; reward *any* defensible choice with a correct justification, then reveal the intended answer. 4. Ask each pair to invent their own odd-one-out set for the next lesson's starter. **Answers / expected outcomes:** (a) **PNG** — lossless; the rest are lossy. (b) **JPEG** — lossy; the rest are lossless techniques/formats. (c) **hash digest** — output of a one-way function, not a key at all; the others are encryption keys. (d) **key pair** — belongs to asymmetric encryption; the other three describe symmetric. Accept alternatives with sound reasoning (e.g. in (c) "session key — the only symmetric key"). **Stretch:** Ask pairs to find a *second* defensible odd one out in each set and justify it.

Always–Sometimes–Never: Compression, Encryption & Hashing (always-sometimes-never · 12 min · threes)

Resources: Three labelled corners of the room (or A/S/N cards); statements on board **How it runs:** 1. Reveal one statement at a time; groups of three discuss for 30 seconds, then move to the Always / Sometimes / Never corner (or hold up a card). 2. Statements: (a) "Applying RLE makes a file smaller." (b) "Lossless compression lets you recover the exact original file." (c) "Lossy compression is appropriate for a text file." (d) "A hashed password can be converted back into the original password." (e) "Ciphertext can be turned back into plaintext by someone holding the correct key." (f) "The public key must be kept secret." (g) "Two different files can produce the same hash value." 3. Pick one student from the majority corner and one from a minority corner to argue; class may switch corners after hearing both. 4. Confirm and correct misconceptions after each statement. **Answers / expected outcomes:** (a) **Sometimes** — only with long runs; alternating data grows. (b) **Always** — the definition of lossless. (c) **Never** — losing characters corrupts text/code. (d) **Never** — hashing is one-way. (e) **Always** — encryption is reversible with the key (that is the point of a key). (f) **Never** — the public key is shared openly; the *private* key is secret. (g) **Sometimes** — a collision; good hash functions make it rare but it is possible because infinite inputs map to fixed-length outputs. **Stretch:** Groups rewrite each "Sometimes" statement so it becomes an Always, and each "Never" so it becomes true.

1.3.2 Databases

Normalisation Card Sort: From Chaos to 3NF (card sort · 20 min · groups of 3–4)

Resources: Per group: ~20 strips of card/paper (students can write them in step 1); table headers on sticky notes **How it runs:** 1. Give each group this unnormalised school record read from the board; they copy each field value onto separate cards: *Student 101, Ada Lovelace, tutor group 12A, takes CS01 "Computer Science" with Mr Khan in room T5 AND MA02 "Maths" with Mrs Cole in room T9. Student 102, Alan Turing, 12B, takes CS01 "Computer Science" with Mr Khan in T5.* 2. **To 1NF:** groups physically lay the cards out as one flat table with no repeating groups — one row per student-per-course — and agree a primary key. Ask: "What data appears more than once?" 3. **To 2NF:** groups split cards into separate physical tables so that no attribute depends on only *part* of the composite key. Prompt: "Does `Name` depend on the `CourseID`? Does `CourseName` depend on the `StudentID`?" 4. **To 3NF:** groups hunt the transitive dependency ("Does `Room` really depend on the course — or on the teacher?") and split again. 5. Groups walk to another table and peer-check: every table has a PK, and "the key, the whole key, and nothing but the key." **Answers / expected outcomes:** **1NF:** `Enrolment(StudentID, Name, TutorGroup, CourseID, CourseName, Teacher, Room)` with composite PK (`StudentID, CourseID`); 3 rows: 101/CS01, 101/MA02, 102/CS01 — students should spot Ada's name written twice and Mr Khan/T5 twice (redundancy). **2NF:** `Student(StudentID, Name, TutorGroup)`, `Course(CourseID, CourseName, Teacher, Room)`, `Enrolment(StudentID, CourseID)` — `Name/TutorGroup` depended only on `StudentID`; `CourseName/Teacher/Room` only on `CourseID` (partial dependencies removed). **3NF:** split `Course` into `Course(CourseID, CourseName, TeacherID)` and `Teacher(TeacherID, TeacherName, Room)` — `Room` depends on `Teacher` (a non-key attribute), a transitive dependency. Expected justification: less redundancy, no update/insert/delete anomalies. **Stretch:** Add a card "Student 101 also takes CS01 in a second lesson slot" and ask whether the current PK still works (needs the slot in the key).

SQL A&E: Spot and Fix (spot-and-fix errors · 12 min · pairs, mini-whiteboards)

Resources: Mini-whiteboards; six broken queries on board/slide **How it runs:** 1. Explain the schema on the board: `Student(StudentID, Name, Age, TutorGroup)`. 2. Reveal one "injured" query at a time. Pairs diagnose the error(s) and write the corrected query on a mini-whiteboard; hold up on a count of three. 3. Queries: (a) `SELECT Name, FROM Student WHERE Age > 16` (b) `SELECT * FROM Student WHERE Name = Amir` (c) `SELECT Name FROM Student ORDER BY Age WHERE Age > 16` (d) `DELETE Student WHERE StudentID = 12` (e) `SELECT Name FROM Student WHERE Name LIKE '%S'` (*intended: names beginning with S*) (f) `UPDATE Student SET Age = 17 StudentID = 3` (*intended: only student 3*). 4. For each, ask a pair to explain what would happen if the broken version ran (error, or worse — the wrong rows changed). **Answers / expected outcomes:** (a) stray comma: `SELECT Name FROM Student WHERE Age > 16`. (b) missing quotes: `... WHERE Name = 'Amir'`. (c) clause order — `WHERE` before `ORDER BY`: `SELECT Name FROM Student WHERE Age > 16 ORDER BY Age`. (d) missing `FROM`: `DELETE FROM Student WHERE StudentID = 12`. (e) wildcard on wrong side — `'%S'` means *ends with S*; fix: `LIKE 'S%'`. (f) missing `WHERE` keyword: `UPDATE Student SET Age = 17 WHERE StudentID = 3` — and highlight that `UPDATE ... SET Age = 17` with no `WHERE` would age the entire school. Emphasise the order `SELECT ... FROM ... WHERE ... ORDER BY`. **Stretch:** Pairs write one deliberately broken `JOIN ... ON` query for the class to fix.

The £70 Bank Job: ACID Role Play (unplugged role play · 15 min · whole class)

Resources: ~20 counters/cubes (or paper "pound coins"); board as the "ledger" **How it runs:** 1. Two students are accounts: **A holds £50, B holds £20** in counters (total £70 on the board — the *consistency*

rule: total must always be £70). 2. A "teller" student performs **TRANSFER £30 A→B** in two steps: take £30 from A... teacher shouts "**POWER CUT!**" before the deposit. Where is the money? Class decides what the database must do → put it back (**rollback** — Atomicity: all-or-nothing). 3. Re-run successfully; the scribe writes the committed balances (£20/£50) permanently on the board ledger. Teacher "crashes" the room again (everyone wipes whiteboards) — the ledger survives (**Durability**). 4. **Lost update round**: two tellers *both* read A's balance as £50. Teller 1 withdraws £10 and writes 40; Teller 2 (who still believes 50) withdraws £20 and writes 30. Ask: what should A hold? What does it hold? 5. Re-run with **record locking**: Teller 2 must wait (arms crossed) until Teller 1 commits. Debrief all four ACID letters + Isolation shown by the lock. **Answers / expected outcomes**: Power-cut round: without rollback the system holds £40 + £20 = £60 — £10 vanished and consistency (total £70) is broken; atomicity forces a full rollback to £50/£20. Lost update round: correct final balance = 50 – 10 – 20 = **£20**, but the unlocked run ends at **£30** because Teller 2's write overwrites Teller 1's — the lost update problem, prevented by record locking. Durability: committed data survives failure. **Stretch**: Stage a deadlock — Teller 1 locks account A and waits for B while Teller 2 locks B and waits for A — and ask how the DBMS should escape (abort/roll back one transaction).

Which Key Am I? Whiteboard Drill (whiteboard drill · 10 min · individual)

Resources: Mini-whiteboards; schema on board **How it runs**: 1. Put a library schema on the board: `Student(StudentID, Name, TutorGroup), Book(BookID, Title, ISBN), Loan(StudentID, BookID, DateOut)`. 2. Fire quick questions; students write PK / FK / composite / secondary (with table names) and hold up: (a) "Primary key of Student?" (b) "Primary key of Loan?" (c) "Name every foreign key in the schema." (d) "A librarian often searches by Title — what should Title become?" (e) "Why is the Loan table there at all?" (f) "Loan contains StudentID 999 but Student doesn't — what rule is broken?" 3. After each reveal, one student justifies before the answer is confirmed. **Answers / expected outcomes**: (a) StudentID. (b) The **composite key** (StudentID, BookID, DateOut) — StudentID+BookID alone fails if the same student reborrows the same book on a later date. (c) In Loan: StudentID → Student.StudentID and BookID → Book.BookID (a foreign key is the primary key of *another* table — name the table). (d) A **secondary key** (indexed non-primary attribute for fast searching). (e) It is the **linking table** resolving the many-to-many between Student and Book into two one-to-many relationships. (f) **Referential integrity** — every FK must reference an existing PK (or be null); 999 is an orphaned record. **Stretch**: Ask students to redesign the schema so a book can have multiple physical copies (introduce Copy(CopyID, BookID) and re-route the FK).

Always–Sometimes–Never: Databases (always-sometimes-never · 10 min · pairs)

Resources: A/S/N cards or three corners; statements on board **How it runs**: 1. Reveal statements one at a time; pairs commit to Always, Sometimes or Never, then defend. 2. Statements: (a) "A primary key uniquely identifies each record." (b) "A primary key is a single attribute." (c) "A foreign key is the primary key of another table." (d) "A table in 3NF is also in 1NF." (e) "A fully normalised database contains no repeated values anywhere." (f) "Two transactions can simultaneously edit the same locked record." (g) "Redundancy in a system is a bad thing." 3. For each, a pair from each camp gives their best one-sentence case before the reveal. **Answers / expected outcomes**: (a) **Always** — by definition. (b) **Sometimes** — composite keys use 2+ attributes. (c) **Always** — that is what makes it foreign. (d) **Always** — the forms are cumulative ($3NF \subseteq 2NF \subseteq 1NF$ conditions). (e) **Never** — foreign key *values* necessarily repeat to create links; normalisation removes *redundant* duplication, not linking values. (f) **Never** — the lock exists precisely to serialise access (Isolation). (g) **Sometimes** — the trap: *data* redundancy is bad (anomalies), but *hardware/backup* redundancy (RAID, failover) is deliberately good. **Stretch**: Pairs write one new "Sometimes" statement whose truth hinges on a definition (e.g. involving flat files or ACID).

1.3.3 Networks

Packet Post: Packet-Switching Role Play (unplugged role play · 20 min · whole class)

Resources: 4 envelopes per message; marker pens; desks arranged so 4–5 "router" students form a diamond with multiple paths **How it runs:** 1. Cast: 1 **sender**, 1 **receiver** (opposite corners), 4–5 **routers** in between arranged so there are at least two routes, and the rest as observers/next-round players. 2. Sender writes the message **MEET ME AT NOON** split across 4 envelopes, one word each. On each envelope: **destination address, source address, sequence number (e.g. 2 of 4)** — the header. 3. Sender hands envelopes, one at a time, to *any* adjacent router. Each router reads only the destination and passes the packet to whichever neighbour is free/closer — packets may take different routes. 4. Secretly instruct one router to sit on packet 2 for 20 seconds ("congestion"). Packets arrive out of order: 1, 3, 4 ... then 2. Receiver must hold them and reassemble by sequence number before reading the message aloud. 5. Round 2: teacher confiscates a packet entirely ("packet loss"). Receiver notices the gap in sequence numbers and sends a "resend packet 3" slip back — retransmission (TCP-style reliable delivery). 6. Debrief: why did the message still arrive intact? What did the header do? Contrast with **circuit switching**: 4 students hold hands forming one reserved path from sender to receiver; nobody else may use those students until the whole call ends. **Answers / expected outcomes:** Students should articulate: data is split into **packets**, each with header (source/destination address, sequence number); packets are **routed independently** and may arrive **out of order**; sequence numbers allow **reassembly**; lost packets are detected and **retransmitted**; the network is efficient and resilient (a busy/failed router is simply routed around). Circuit switching contrast: dedicated path, guaranteed capacity, but reserved links sit idle and a broken link kills the call. **Stretch:** Add a second sender/receiver pair transmitting simultaneously through the same routers — packet switching interleaves both conversations on shared links; circuit switching cannot.

The TCP/IP Post Office (jigsaw teaching + analogy build/critique · 20 min · groups of 4)

Resources: Sugar paper or whiteboards per group; layer-name cards **How it runs:** 1. **Expert phase:** split the class into four expert stations, one per TCP/IP layer (Application, Transport, Network/Internet, Link), each with the knowledge organiser row for their layer. Each station's job: decide what their layer is in a postal system, and which protocols live there. 2. **Jigsaw phase:** re-form into mixed groups of four (one expert per layer). Each group builds the full "post office" analogy on sugar paper, tracing one letter from writer to reader — down the stack at the sender, up the stack at the receiver — showing what gets *added* at each layer on the way down and *removed* on the way up. 3. Target mapping to steer towards: Application = writing the letter using agreed conventions (HTTP/SMTP = "how a letter is laid out"); Transport = splitting a long letter into numbered envelopes, checking all arrived, requesting re-sends, and delivering to the right *person* in the building (TCP, ports); Network = writing the building's address and sorting offices choosing the route (IP addresses, routers); Link = the van/postal worker physically carrying mail between *adjacent* sorting offices (MAC addresses, Ethernet/Wi-Fi). 4. **Critique phase:** each group must find and present at least two places the analogy *breaks down*. 5. Gallery walk; class votes for the most accurate poster. **Answers / expected outcomes:** Posters show 4 layers in order, correct protocol placement (HTTP/HTTPS/FTP/SMTP/POP3/IMAP/DNS → Application; TCP/UDP → Transport; IP → Network; Ethernet/Wi-Fi → Link), headers added going down and stripped going up, and the key subtlety: the **IP address (destination building) stays constant end-to-end while the "van" (MAC address) changes at every hop**. Valid critiques include: real post doesn't automatically reorder pages or acknowledge receipt (TCP does); one letter isn't literally wrapped in nested envelopes at each layer (headers are); routers inspect every packet, sorting offices don't read your letter. **Stretch:** Groups

annotate where a firewall and a proxy server would sit in the postal analogy (security checkpoint / mailroom that forwards, filters, and caches on your behalf).

DNS Detectives (unplugged role play · 12 min · whole class)

Resources: 4 cards prepared live: "Browser", "Local DNS resolver + cache", "DNS hierarchy directory", "Web server 93.184.216.34" **How it runs:** 1. Cast: a **browser** student, a **resolver** student holding a blank "cache" whiteboard, a **directory** student across the room holding the master list (`schoolsite.co.uk` = 93.184.216.34), and a **web server** student wearing the IP number. 2. Browser wants `schoolsite.co.uk`. It asks the resolver. Cache is empty (miss), so the resolver must walk across the room to query the directory (narrate: in reality this is a *hierarchy* — root, then `.uk`, then the domain's own name server). 3. Resolver writes the answer into its cache, returns the IP; browser can now contact the web server directly using the IP address. Time the whole lookup dramatically with a stopwatch. 4. A second browser student requests the same site. Resolver answers instantly from cache — compare times. 5. Debrief: why do we have DNS at all? Why cache? What breaks if the DNS server is down?

Answers / expected outcomes: Students articulate: humans use memorable domain names, routers need **IP addresses**, and **DNS translates names → IPs**; the system is **hierarchical** (queries walk down levels) and **cached** (repeat lookups are far faster and reduce load); without DNS you could still reach a site by typing its raw IP. Lookup 2 should be visibly ~instant vs lookup 1's walk. **Stretch:** Poison the cache (resolver secretly writes the wrong IP) and discuss why the browser trusts it — and what that implies for security.

Protocol Corners (decision corners card sort · 10 min · whole class)

Resources: Five corner/wall signs: HTTP(S), FTP, SMTP, POP3, IMAP **How it runs:** 1. Stick the five protocol signs around the room. 2. Read scenarios one at a time; students walk to the protocol that fits, then one student per corner justifies before the reveal: (a) "A developer uploads new website files to the web server." (b) "Your mail app sends a finished email to the mail server." (c) "You read your inbox on your phone at lunch and the same inbox, still in sync, on your laptop at home." (d) "An old desktop program downloads mail to one PC and deletes it from the server." (e) "You view your online banking dashboard — the padlock icon is showing." (f) "Your browser requests a plain, unencrypted web page." 3. After each, ask a mislocated student what feature would have signposted the right answer. **Answers / expected outcomes:** (a) **FTP** (file transfer). (b) **SMTP** (sending mail — client → server and server → server). (c) **IMAP** (mail stays on the server; multiple synced devices). (d) **POP3** (downloads and removes; single device). (e) **HTTPS** (HTTP encrypted with TLS/SSL). (f) **HTTP**. Discriminator to draw out: SMTP *sends*; POP3/IMAP *retrieve* — and IMAP vs POP3 hinges on whether mail remains on the server. **Stretch:** Add ambiguous scenario "your phone notifies you of a new email" and debate the full chain (sender's SMTP → recipient server, then IMAP retrieval).

Network Design Continuum (decision continuum · 12 min · pairs then whole class)

Resources: Masking-tape line across the room labelled "Client-server" at one end, "Peer-to-peer" at the other; scenario list **How it runs:** 1. Pairs hear a scenario, discuss for 30 seconds, then stand along the continuum to show their recommendation (middle = "genuinely mixed"). 2. Scenarios: (a) a 1,200-student school network with shared files and central logins; (b) two flatmates sharing a printer and files between their laptops; (c) a bank branch network; (d) a large-scale file-sharing network with no central server wanted; (e) a small LAN party of 6 gamers with no budget. 3. Interview students at extreme and middle positions: "What would change your mind?" 4. Debrief the trade-off table: security, backup, cost, single point of failure, scalability. **Answers / expected outcomes:** (a) **Client-server** — central security, backups and user management at scale. (b) **P2P** — cheap, trivial to set up, nothing critical. (c) **Client-server** —

security/auditability essential. (d) **P2P** — decentralised by requirement; no single point of failure. (e) **P2P** — no server cost; acceptable management burden. Reasoning must reference: client–server gives central management/backup/security but the server is a single point of failure and costs more; P2P is cheap and resilient but harder to secure and manage. **Stretch:** Re-run scenario (b) five years later when the flat becomes a 40-person startup — at what size does the answer flip, and why?

1.3.4 Web Technologies

HTML, CSS or JavaScript? Card Sort (card sort · 12 min · pairs)

Resources: Per pair: ~15 slips (project the list; pairs write slips themselves — 2 min) **How it runs:** 1. Pairs make three column headers: **HTML — structure, CSS — presentation, JavaScript — behaviour**. 2. They sort these 15 slips: `<h1>Results</h1>` · `color: red;` · `alert("Saved!")` · `` · `#menu { width: 200px; }` · a click event handler · `<a href="page2.html">Next</a>` · choosing the page font · checking a form field contains "@" before submission · `.warning { font-weight: bold; }` · `<p>` tags · updating the page content without reloading · `<div id="header">` · centring the layout · responding when the user presses a key. 3. Pairs then place a fourth "wildcard" header — **server-side** — and decide if *any* slip could belong there instead. 4. Reveal and resolve disputes; finish with "why do we separate the three at all?" **Answers / expected outcomes:** HTML: `<h1>`, `<img>`, `<a href>`, `<p>`, `<div id="header">`. CSS: `color: red;`, `#menu {...}` (id selector), `.warning {...}` (class selector), font choice, centring layout. JavaScript: `alert()`, click handler, "@" validation, dynamic updates, keypress response. Wildcard discussion: client-side "@" validation improves user experience but must be *repeated server-side* because client-side code can be viewed and bypassed. Separation = maintainability: restyle without touching structure or behaviour. **Stretch:** Give the slip `<script>` and `<link rel="stylesheet">` — where do the three technologies actually live inside one HTML file?

Human PageRank (unplugged human demonstration · 15 min · whole class)

Resources: 36 counters (or squares of paper); 3 large page signs A, B, C; string/arrows on floor **How it runs:** 1. Three students are web pages holding signs. Lay string arrows on the floor: **A links to B and C; B links to C; C links to A**. 2. Deal every page 12 counters (equal starting rank; 36 total in the "web"). 3. **Iteration rule:** on each round, every page splits its *entire* pile equally between its outgoing links and passes it along the string, all simultaneously. 4. Run round 1; pause and record piles on the board. Run round 2; record again. 5. Ask: who's winning and *why*? What is a link "worth"? Keep iterating a round or two and observe the values drifting rather than exploding — introduce the words **iterative** and **converge**, and mention the real algorithm adds a **damping factor** (~0.85, the "bored random surfer") which our floor model omits. **Answers / expected outcomes:** Start: A=12, B=12, C=12. **Round 1:** A sends 6 to B and 6 to C; B sends 12 to C; C sends 12 to A → A=12, B=6, C=18. **Round 2:** A sends 6+6; B sends 6 to C; C sends 18 to A → **A=18, B=6, C=12** (total always 36). Key ideas: a link is a *vote*; a vote from a high-rank page is worth more (C's single link makes A rich); a page **divides** its rank across all its outgoing links; ranks are recomputed **iteratively until they converge**. PageRank measures *importance from incoming links*, not keyword relevance. **Stretch:** Add a page D that links to C but has no incoming links — predict D's fate over iterations (its rank drains away; in the real algorithm damping keeps it small but non-zero).

Crawl, Index, Answer (unplugged role play · 15 min · whole class)

Resources: 5 A4 "web pages" (write live or pre-scribble): each has a title, 3–4 keywords, and links to 1–2 other pages; stick around the room **How it runs:** 1. Prepare pages, e.g. P1 "Binary Basics" (keywords: binary, bits; links → P2, P3), P2 "Hex for Humans" (hex, binary; → P4), P3 "Logic Gates" (AND, binary; →

P4), P4 "Networks 101" (packet, router; → P5), P5 "Router Buying Guide" (router; no links). 2. One student is the **crawler**: starting at P1, they physically follow links, and at each page shout out its keywords. They may only travel along links. 3. Two students are the **indexers** at the board, building an inverted index as the crawler shouts: keyword → list of pages. 4. Now the class becomes users: "Search: *binary*!" The indexers answer instantly from the board. Contrast: ask a fresh student to answer "which pages mention *router*?" by physically re-walking the whole web — time both. 5. Debrief: crawler follows links; index maps keywords → pages; queries hit the *index*, never the live web; ranking (e.g. PageRank) then orders the results. **Answers / expected outcomes:** Final index: binary → P1, P2, P3; bits → P1; hex → P2; AND → P3; packet → P4; router → P4, P5. Query "binary" returns P1, P2, P3 instantly from the index; the manual re-walk is visibly slower — the point of indexing. Students should also notice the crawler only found P2–P5 *because links existed*: unlinked pages never get indexed. **Stretch:** Ask which of P4 and P5 should rank first for "router" and defend using incoming links (P5 has a link from P4; P4's link comes from P3 — bring in PageRank reasoning).

Broken Homepage: Spot and Fix (spot-and-fix errors · 10 min · pairs, mini-whiteboards)

Resources: Snippet on board/slide; mini-whiteboards **How it runs:** 1. Display this "intern's homepage" and announce there are **five** errors — pairs list line numbers + fixes on whiteboards:

```
<html>
<head><title>My Page</head></title>
<style>.big { colour: red }</style>
<body>
  <h1>Welcome<h1>
  <img href="cat.jpg">
  <a src="page2.html">Next page</a>
</body>
</html>
```

2. Pairs hold up boards; tally which errors were most-spotted and least-spotted.

3. A volunteer pair rewrites the corrected version on the board; class verifies against the five.

Answers / expected outcomes: (1) `</head></title>` wrongly nested → `</title></head>`. (2) CSS property `colour` is invalid → `color: red`. (3) `<h1>Welcome<h1>` — second tag isn't a closing tag → `</h1>`. (4) `<img href=...>` — images use `src` → ``. (5) `<a src=...>` — anchors use `href` → `<a href="page2.html">`. Discussion point: browsers often render broken HTML anyway ("forgiving"), which makes these bugs easy to ship. **Stretch:** Ask pairs to add a class attribute so the heading actually uses `.big`, and an id + matching `#` CSS rule.

Client-Side vs Server-Side Continuum (decision continuum + think-pair-share · 12 min · whole class)

Resources: Tape line: "Client-side" ↔ "Server-side"; scenario list **How it runs:** 1. Think (30s) – pair (30s) – then students stand on the line for each task; middle = "needs both". 2. Tasks: (a) highlighting a menu button when the mouse hovers; (b) checking the password typed matches the stored password hash; (c) warning the user that the email field is empty before submitting; (d) searching the product database for "trainers"; (e) validating a card payment; (f) form validation on a banking site. 3. Interview outliers each round: "what happens to the server if we move this to the client — and what happens to security?" 4. Board the trade-off summary: speed/responsiveness & reduced server load vs security & access to the

database. **Answers / expected outcomes:** (a) **Client** — instant response, no data needed from server. (b) **Server** — the hash and comparison must stay hidden; client code can be read and bypassed. (c) **Client** (for UX) — but re-checked server-side. (d) **Server** — the database lives there. (e) **Server** — security-critical. (f) **Both** — the exam-grade answer: client-side validation gives fast feedback and cuts server load, but is *not secure* because users can view/modify/bypass browser code, so the server must validate again. **Stretch:** One-liner to debate: "If server-side is more secure, why not do *everything* server-side?" (latency, server load, bandwidth, offline responsiveness).